

PRACTICAL – 7

PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

AIM:

To process the StudentHub registration form using PHP, validate and sanitize inputs on the server, store accepted registration requests in CSV and JSON files, and display clear success and error messages. The practical also implements duplicate-email checking, a records-view page and session-based CSRF protection.

Introduction:

The existing StudentHub registration form was connected to a PHP backend running through XAMPP Apache. JavaScript performs client-side checks and submits FormData using fetch(). PHP independently checks the submitted values before saving them. Valid requests are stored as structured JSON records and CSV rows; invalid requests return field-specific errors without adding a registration.

Tools and Technology:

VS Code, PHP, XAMPP Apache, HTML5, CSS3, JavaScript, Fetch API, FormData, JSON, CSV, PHP sessions, browser developer tools, Git and GitHub.

Files Used:

File	Purpose
register.html / js/validation.js	Registration form, client validation and POST submission.
php/get-csrf-token.php	Generates a token and stores it in the PHP session.
php/process-registration.php	Validates inputs, checks duplicates and writes records.
registrations.html / js/registrations.js	Displays saved registration records.
php/storage/registrations.json	Structured registration records.
php/storage/registrations.csv	Header row followed by registration rows.

Key Skills Used:

POST handling with \$_POST; validation with regular expressions, filter_var() and allowlists; sanitization with trim(), strip_tags() and htmlspecialchars(); JSON encoding and decoding; CSV writing with fputcsv(); exclusive file locks with flock(); and CSRF checking with PHP sessions and hash_equals().

Implementation Overview:**1. Form Submission using POST**

The submit handler prevents the default form action, validates the inputs locally and sends new FormData(form) to the PHP processing endpoint with method: POST. PHP rejects other HTTP methods with a 405 response.

2. Server-Side Validation and Sanitization

PHP checks the name length, email format, 10-digit mobile number, password rules, matching confirmation, course, year, gender and acceptance of terms. String inputs are checked before use. Names are trimmed, stripped of HTML tags and escaped before storage. Invalid inputs produce an errors object with the relevant field keys.

3. Duplicate-Email Checking

The existing JSON records are read while holding an exclusive lock. strcasecmp() compares the submitted email with saved emails without case sensitivity. A duplicate is rejected before a new record is written.

4. CSV and JSON Storage

Accepted requests contain an ID, name, email, mobile number, course, year, gender, role and timestamp. JSON stores an array of records. CSV stores a header and one row per request. Password values are not saved; this practical records account requests rather than implementing login authentication.

5. Checked File Handling

Folder creation, file opening, locking, reading, encoding and writing are checked. Malformed existing JSON is rejected instead of discarded. Both files are locked before writing; detected failures trigger restoration attempts and clear errors. Separate file writes are not a crash-proof transaction.

6. Success and Error Responses

PHP returns JSON containing success, message and errors. JavaScript displays success or error messages and marks fields with server errors. A successful submission resets the form and requests a fresh token.

7. Advanced Extension — CSRF Protection and Records View

The browser requests a session token from get-csrf-token.php and includes it as csrf_token. PHP uses hash_equals() to verify it before validation and storage. An invalid token returns 403. The Registrations page displays saved records and was checked after refresh.

Testing Summary:

Test	Result
Valid registration and JSON/CSV storage	Passed; rechecked after the PHP update (user confirmed).
Duplicate email	Rejected with a field error; no additional record.
Direct POST with mobile = 123	Rejected by PHP despite bypassing client validation.
Invalid CSRF token	403 Forbidden and success: false.
Records view after refresh	Passed (user confirmed); screenshot to be inserted.
Simulated file-write failure	Not executed; failure handling added in code.

Screenshots:

The following screenshots were captured during the practical. The storage images show the initial successful record; the later successful save after the file-handling update was confirmed separately.

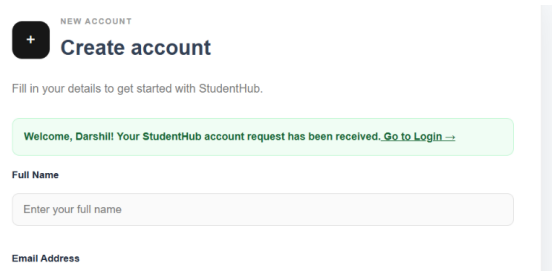


Figure 1: Successful registration response; the form resets after PHP accepts the request.

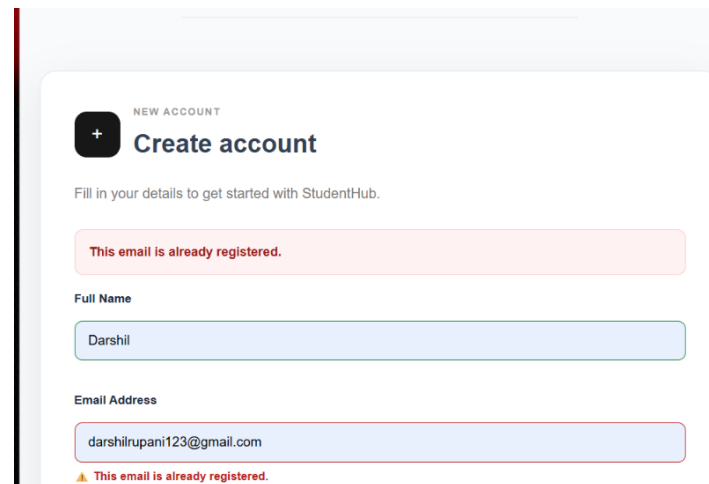


Figure 2: Duplicate-email rejection displayed at the top of the form and below the email field.

```
php > storage > {} registrations.json > ...
1  [
2    {
3      "id": "reg_6ac5271731f9e9.39791906",
4      "fullname": "Darshil",
5      "email": "darshilrupani123@gmail.com",
6      "mobile": "6792523392",
7      "course": "btech-ce",
8      "year": "3",
9      "gender": "male",
10     "role": "student",
11     "password_set": true,
12     "registered": "2026-10-06T18:51:35+02:00"
13   }
14 ]
```

Figure 3: registrations.json showing one saved registration record.

```
php > storage > [] registrations.csv
1  id,fullname,email,mobile,course,year,gender,role,registered
2  reg_6ac5271731f9e9.39791906,Darshil,darshilrupani123@gmail.com,6792523392,btech-ce,3,male,student
3
```

Figure 4: registrations.csv showing the header and one saved registration row.

Screenshots — Server Validation and CSRF:

```
data.set('terms', 'on');

const response = await fetch('php/process-registration.php', {
  method: 'POST',
  body: data
});

console.log(JSON.stringify(await response.json(), null, 2));
})();
```

< ▶ Promise {<pending>}

```
{
  "success": false,
  "message": "Please fix the errors below and try again.",
  "errors": {
    "mobile": "Mobile number must be 10 digits, starting with 6, 7, 8 or 9."
  }
}
```

>

Console What's new X AI assistance Issues

Figure 5: Direct POST test with mobile number 123; PHP returns success: false and a mobile validation error.

Elements Console Sources Network >> 1

top Filter Default levels No issues

```
> fetch('php/process-registration.php', {
  method: 'POST',
  body: new URLSearchParams({ csrf_token: 'invalid-token' })
})
.then(response => response.json())
.then(data => console.log(JSON.stringify(data, null, 2)));
```

< ▶ Promise {<pending>}

✖ ▶ POST VM122:1

http://localhost/studenthub-portal/php/process-registration.php 403 (Forbidden)

```
{
  "success": false,
  "message": "Your session expired. Please reload the page and try again.",
  "errors": []
}
```

>

Console What's new X AI assistance Issues

New in DevTools (October 2026)

See all new features

DevTools for agents

on anything GPT-6.1 Sol Medium

Figure 6: Invalid CSRF token rejected with HTTP 403 and the session-expired message.

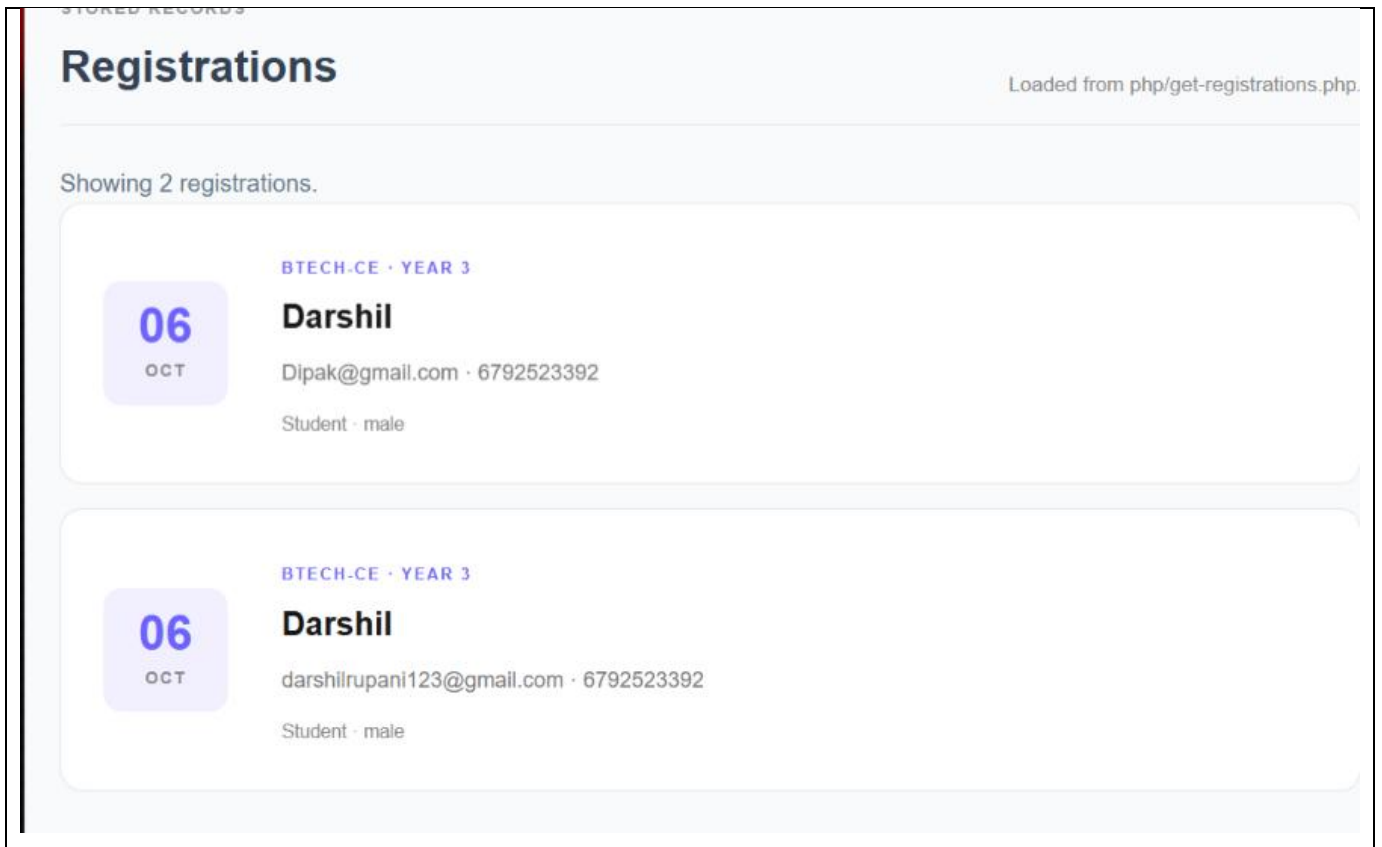
Screenshots — Records View and Source Submission:

Figure 7: Registrations page displaying records retrieved from storage.

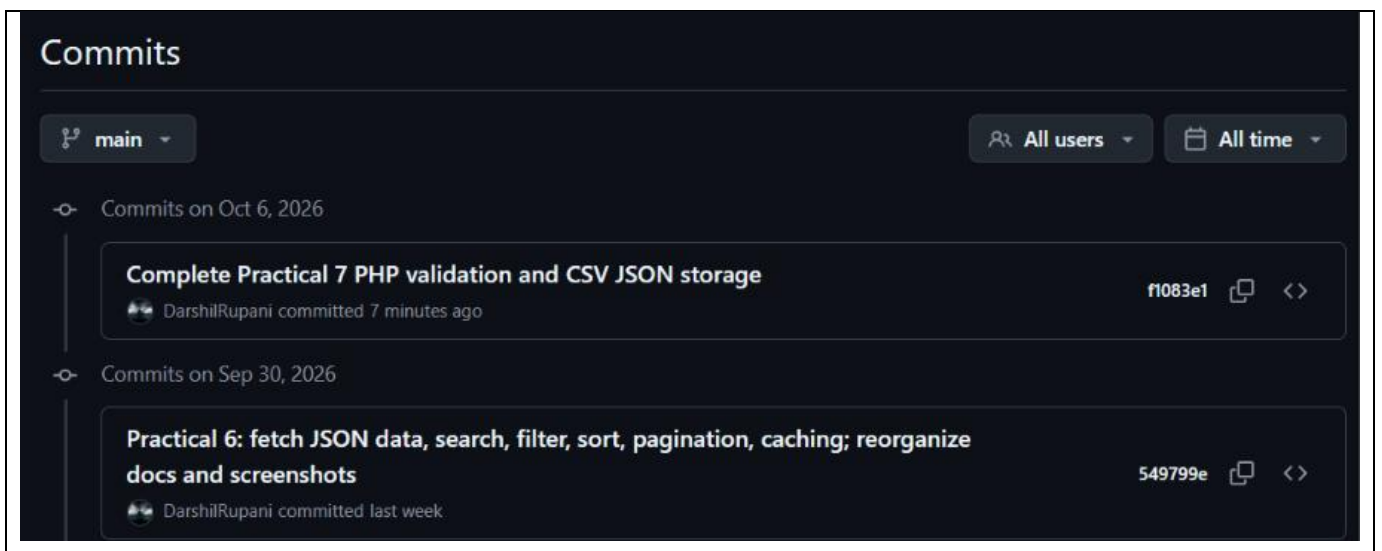


Figure 8: Practical 7 source-code submission on GitHub.

GITHUB REPOSITORY LINK:

<https://github.com/DarshilRupani/studenthub-portal>

(FOR SOURCE CODE)

Practical 7 source pushed to main: commit f1083e1 — Complete Practical 7 PHP validation and CSV JSON storage. Push verified from terminal output.

Key Questions and Answers**Q1. Is the form submitted using POST?**

Yes. JavaScript sends the form using `fetch()` with method: POST and a `FormData` body. PHP reads `$_POST` and checks `REQUEST_METHOD`; requests using other methods are rejected.

Q2. Are inputs validated and sanitized server side?

Yes. PHP validates required fields, email, mobile, password and confirmation, course, year, gender and terms. `trim()` removes surrounding spaces, `strip_tags()` removes HTML tags from the name, and `htmlspecialchars()` escapes special characters in the stored name. `filter_var()` validates the email, and allowlists restrict dropdown and radio values. These checks run independently of JavaScript.

Q3. Is file writing handled safely?

The processing code checks folder creation, file opening, locks, reads, encoding, writes and flushes. Exclusive locks protect updates made through this endpoint. Existing invalid JSON is not silently overwritten. Detected failures trigger restoration attempts; incomplete restoration is reported. JSON and CSV are separate files, so a process crash between writes is still a limitation.

Q4. Are success and error responses displayed clearly?

Yes. PHP returns success, message and errors as JSON. JavaScript displays a success message after saving, a general error message for rejected requests and field-level messages for validation errors. Invalid CSRF tokens return HTTP 403, while storage errors return HTTP 500.

Post Laboratory Work:

Submit the PHP source files, test cases and generated CSV/JSON files along with this report. The report includes evidence of successful submission, duplicate rejection, storage, direct server-side validation and CSRF rejection. Insert the records-view and GitHub screenshots in the labeled spaces before final submission.

Conclusion:

Practical 7 connected the StudentHub registration form to PHP processing through XAMPP. Valid account requests are saved in CSV and JSON, while invalid inputs, duplicate emails and invalid CSRF tokens are rejected with clear messages. File operations now include error checks and restoration attempts. Successful submission and storage were rechecked after the update; simulated write failures remain untested.